

Writeup Report — PA2

Student ID: 243121

Name: Aisha Tursynbay

Variant Parameters:

- **Solid Object:** Triangular Prism (from PA1)
- **Orbit Period (T):** 7 seconds ($6 + (1 \bmod 3) = 7s$)
- **Cube Axis of Rotation:** Normalized $[1, 1, 1]$ ($2 \bmod 3 = 2$)
- **Orbit Plane:** Horizontal / Rotation around Y-axis ($0 \bmod 3 = 0$)
- **Camera View:** Eye $[0, 2.5, 7]$, FOV 45° ($0 \bmod 2 = 0$)

E1: Order of Transformation Matrices

Question: Explain the exact matrix multiplication order used to build `uModelMatrix` for the orbiting solid and why changing this order breaks the intended motion.

Answer: The model matrix for the orbiting solid is computed using the following multiplication order:

$$M_{\text{solid}} = R_{\text{orbit}}(t) \cdot T_{\text{orbit}} \cdot R_{\text{self}}(t) \cdot S(t)$$

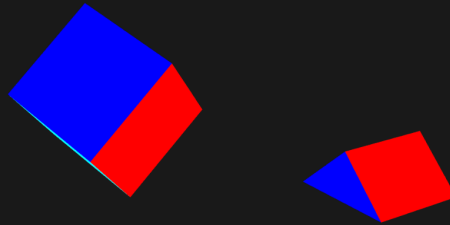
In `glmMatrix` (which applies transformations in post-multiplication order), the code is written as:

1. `mat4.identity(m)`
2. `mat4.rotateY(m, m, orbitAngle)` — Applies orbital rotation around the origin (0,0,0).
3. `mat4.translate(m, m, [2.5, 0, 0])` — Translates the object 2.5 units along the X-axis.
4. `mat4.rotateY(m, m, selfAngle)` — Rotates the object around its local Y-axis (2.0 rad/s).
5. `mat4.scale(m, m, [s, s, s])` — Scales the object around its local origin ($s(t) = 0.65 + 0.15 \sin(2\pi t/3)$).

Why order matters:

- Matrix multiplication is **non-commutative** ($A \cdot B \neq B \cdot A$).
- If `Torbit` were applied *before* `Rorbit`, the rotation would occur around the object's offset center rather than the central Cube, causing the object to spin in place far from the origin instead of orbiting around the Cube.
- Scaling `S(t)` must be applied first (rightmost) so that the scaling happens relative to the object's local origin before translation and orbital rotation.

```
Student ID: 21010321
Proj: Perspective
FOV: 45°
Time (t): 6.1s
FPS: 30
```



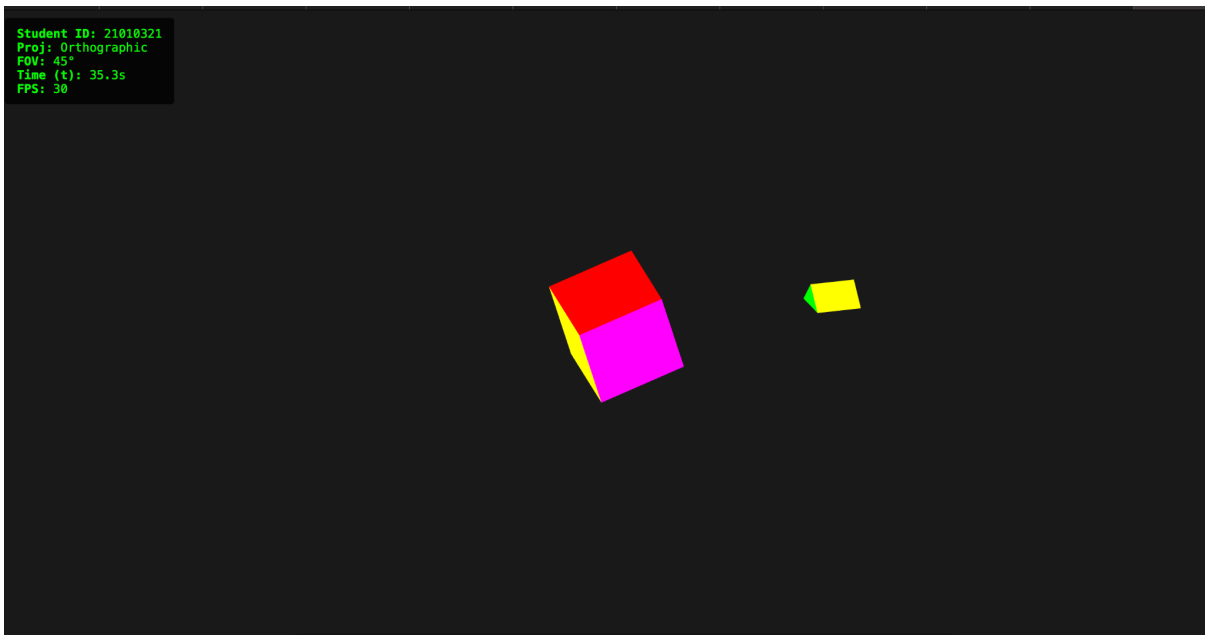
E2: Cube Rotation Implementation

Question: Describe how the Cube's transformation matrix is calculated and how it differs from the solid's matrix.

Answer: The Cube remains stationary at the scene origin (0,0,0) and rotates exclusively around its local axis defined by the variant

Difference from the Solid's matrix:

- **Cube:** Contains **no translation or scaling**, and only rotates around a single fixed, normalized 3D axis [1,1,1] at a constant speed of 1.2 rad/s.
- **Solid:** Combines **4 distinct transformations** (pulsing scale, self-spin, translation radius of 2.5, and orbital revolution).



E4: Frame-Rate Independent Animation

Question: How is time step Δt handled to ensure smooth animation across varying screen refresh rates?

Answer: Animation updates are decoupled from monitor refresh rates by computing the elapsed delta time (Δt) between consecutive frames:

$$\Delta t = \text{currentTime} - \text{previousTime}$$

E5: Single Buffer Strategy and Multiple Draw Calls

Question: How are multiple objects stored in GPU memory and rendered?

Answer: To maximize performance and eliminate redundant GPU buffer bindings, all vertex data for both the Cube (36 vertices) and the Solid (24 vertices) are concatenated into a **single Vertex Buffer Object (VBO)**.

During the render loop:

1. The VBO is bound once.
2. **Cube Draw Call:** The `uModelMatrix` is updated to `Mcube`, and `gl.drawArrays(gl.TRIANGLES, 0, 36)` draws the first 36 vertices.
3. **Solid Draw Call:** The `uModelMatrix` is updated to `Msolid`, and `gl.drawArrays(gl.TRIANGLES, 36, 24)` draws the next 24 vertices using the offset index 36.

E6: User Controls Summary

Question: List the interactive controls implemented in the assignment.

Answer:

- **P:** Toggle pause / resume animation time t .
- **O:** Toggle projection mode between Perspective and Orthographic.
- **+ / -:** Increase / decrease camera FOV by 5° (Perspective mode only).
- **ArrowLeft / ArrowRight:** Orbit the camera horizontally around the scene (Y-axis azimuth angle).
- **R:** Reset time $t=0$, camera orientation, projection mode, and FOV to default settings.